

DEAS: DETACHED VALUE LEARNING WITH ACTION SEQUENCE FOR SCALABLE OFFLINE RL

Changyeon Kim^{1*} Haeone Lee¹ Younggyo Seo² Kimin Lee^{1†} Yuke Zhu^{3,4†}

¹KAIST ²UC Berkeley ³University of Texas at Austin ⁴NVIDIA

ABSTRACT

Offline reinforcement learning (RL) presents an attractive paradigm for training intelligent agents without expensive online interactions. However, current approaches still struggle with complex, long-horizon sequential decision making. In this work, we introduce *DEtached value learning with Action Sequence* (DEAS), a simple yet effective offline RL framework that leverages action sequences for value learning. These temporally extended actions provide richer information than single-step actions and can be interpreted through the options framework via semi-Markov decision process Q-learning, enabling reduction of the effective planning horizon by considering longer sequences at once. However, directly adopting such sequences in actor-critic algorithms introduces excessive value overestimation, which we address through detached value learning that steers value estimates toward in-distribution actions that achieve high return in the offline dataset. We demonstrate that DEAS consistently outperforms baselines on complex, long-horizon tasks from OGBench and can be applied to enhance the performance of large-scale Vision-Language-Action models that predict action sequences, significantly boosting performance in both RoboCasa Kitchen simulation tasks and real-world manipulation tasks.

1 INTRODUCTION

Offline reinforcement learning (RL) (Lange et al., 2012; Levine et al., 2020) enables learning from static datasets without incurring online data collection risks, while circumventing the need for expensive expert demonstrations. However, existing methods primarily focus on short-horizon tasks with dense rewards (Yu et al., 2020; Fu et al., 2020; Gulcehre et al., 2020; Mandelkar et al., 2021) and struggle to scale to complex long-horizon scenarios. Recent attempts using large-scale architectures (Kumar et al., 2023a;b; Chebotar et al., 2023; Springenberg et al., 2024) show promise, but their effectiveness on complex tasks remains unexplored.

To address the need for long-horizon evaluation, recent work (Park et al., 2025a;b) has proposed challenging benchmarks for complex offline RL and demonstrated that reducing the effective planning horizon (i.e., shortening the time span over which the agent must plan) in both value and policy learning via n -step TD updates with high n values and hierarchical policies is essential. However, these approaches rely on goal-conditioned RL with explicit expert-provided goals, which are often unavailable in practice. For instance, high n values in n -step TD updates introduce increased bias and bootstrap error in standard RL without explicit goal information (Tsitsiklis & Van Roy, 1996; Kearns & Singh, 2000; Sutton & Barto, 2018).

These limitations underscore the need for alternative approaches to horizon reduction (reducing the planning horizon) that work without explicit goal conditioning. One promising direction is leveraging action sequences, which have shown success in behavior cloning (Pomerleau, 1988) for capturing noisy, temporally-relevant distributions in expert demonstrations (Chi et al., 2023; Zhao et al., 2023). However, existing attempts to use action sequences for RL remain insufficient for achieving robust horizon reduction. Q-chunking (Li et al., 2025b) has explored the use of action sequences for RL, demonstrating their potential for temporally consistent exploration. However, introducing action sequences to standard actor-critic frameworks causes severe value overestimation (Seo & Abbeel, 2025) due to actors maximizing over potentially erroneous critic estimates with widely spanned

*Work done while visiting University of Texas at Austin.

Project page: <https://changyeon.site/deas>

†Equal advising.

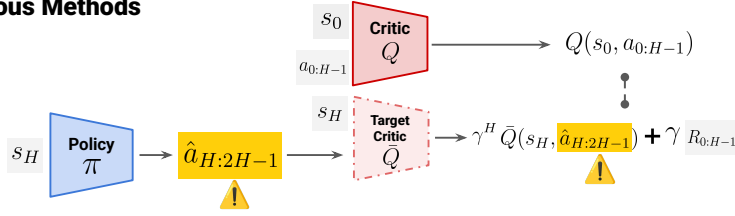
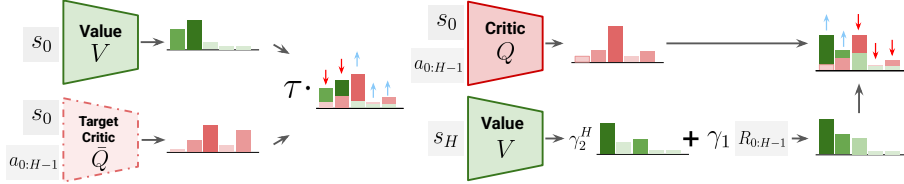
Previous Methods**DEAS (Ours)**

Figure 1: **Overview.** DEAS is an offline RL framework that learns from action sequences instead of single actions. Unlike previous methods that couple actor-critic training, our key insight is to train the critic separately from the policy (detached value learning) using action sequences, which enables stable learning while avoiding value overestimation. We further enhance stability by combining distributional RL objectives and using dual discount factors, which leads to additional improvement.

action spaces. This problem is exacerbated in offline RL where distribution shift creates extrapolation errors (Kumar et al., 2019; Fujimoto et al., 2019; Kumar et al., 2020). While CQN-AS (Seo & Abbeel, 2025) proposes a value-only approach to avoid this issue, it introduces discretization errors that limit performance in complex tasks and cannot leverage expressive policy classes (Wang et al., 2023; Hansen-Estruch et al., 2023; Park et al., 2025c). For this reason, our research aims to develop methods that can leverage action sequences for horizon reduction while avoiding value overestimation and maintaining compatibility with expressive policy architectures.

Our approach We present *DEtached value learning with Action Sequence* (DEAS), an offline RL framework that leverages action sequences for scalable value learning in complex tasks. Our method treats consecutive action timesteps as inputs to the value function, implementing the simplest form of the options framework (Sutton et al., 1999; Stolle & Precup, 2002). This design provides principled horizon reduction analogous to n -step TD updates with temporally extended actions, while action sequences offer richer information than single-step actions without requiring explicit goal conditioning. To address the value overestimation challenges inherent in learning value functions with action sequences in offline RL settings, we employ detached value learning (Kostrikov et al., 2022) that decouples critic training from the actor, biasing value estimates toward high-return actions present in the offline dataset. This method is appealing as it can be applied to any expressive policy architectures including large-scale Vision-Language-Action models (VLAs) without the hazard of value overestimation. Additionally, we propose to incorporate distributional RL (Farebrother et al., 2024) in value learning to mitigate instability from accumulated bias in multi-step returns.

We validate DEAS through comprehensive experiments on challenging long-horizon tasks from OGBench (Park et al., 2025a), where standard offline RL methods struggle to achieve meaningful success rates. Our method consistently outperforms all baselines, demonstrating its effectiveness on complex tasks. Additionally, we show that DEAS can be used to improve the performance of VLAs (Bjorck et al., 2025) in hard tasks from RoboCasa Kitchen (Nasiriany et al., 2024) and real-world manipulation tasks, which significantly improves performance compared to policies trained solely on expert demonstrations. These results demonstrate DEAS’s practical applicability and potential for scaling offline RL to real-world scenarios.

Contributions We highlight the key contributions of our paper below:

- We present DEAS: *DEtached value learning with Action Sequence*, a simple yet effective offline RL method that leverages action sequences for training critics and employs detached value learning with classification loss for stable training.
- We demonstrate that DEAS significantly outperforms baselines on complex, long-horizon tasks across 30 diverse scenarios in OGBench (Park et al., 2025a).

- We demonstrate that DEAS can enhance the performance of large-scale VLAs, achieving superior results on complex tasks from RoboCasa Kitchen (Nasiriany et al., 2024) and real-world manipulation tasks compared to policies trained solely on expert demonstrations.

2 RELATED WORK

Offline reinforcement learning Offline RL focuses on learning policies from fixed datasets without further environment interaction (Levine et al., 2020). The primary challenge lies in the distributional shift between the behavior policy and the learned policy, which can result in value overestimation and suboptimal performance. Previous work has proposed various approaches including weighted regression (Peng et al., 2019; Nair et al., 2020; Wang et al., 2020), conservative regularization (Kumar et al., 2020), behavioral regularization (Fujimoto et al., 2019; Fujimoto & Gu, 2021; Tarasov et al., 2023; Park et al., 2025c), and in-sample distribution maximization (Kostrikov et al., 2022; Xu et al., 2023; Garg et al., 2023). Our method builds upon in-sample distribution maximization approaches, particularly IQL (Kostrikov et al., 2022), extending them to handle action sequences while maintaining stability by removing the critic update based on the actor’s output. Furthermore, our method has the advantage of being adaptable to any policy extraction method for the final policy, making it more flexible and practical.

BC/RL with action sequence Adopting action sequence has been actively investigated in both imitation learning and RL. Behavior cloning advances show that predicting action sequences captures temporal dependencies from expert demonstrations that single-step actions miss (Chi et al., 2023; Zhao et al., 2023; Black et al., 2025; Bjorck et al., 2025; Intelligence et al., 2025). Several works have introduced action sequences into RL (Li et al., 2024; Tian et al., 2025), with Q-Chunking (Li et al., 2025b) demonstrating incorporation into actor-critic frameworks in offline-to-online RL without policy class constraints. However, this approach faces fundamental challenges: expanded action spaces increase value overestimation risk, particularly in offline settings with limited data coverage (Kumar et al., 2019), yet this issue remains unaddressed. CQN-AS (Seo & Abbeel, 2025) circumvents this by removing the actor entirely, but introduces accumulating discretization errors that severely limit performance in complex tasks and prevent use of expressive policy classes (Wang et al., 2023; Park et al., 2025c). Our approach uniquely combines both paradigms: we leverage horizon reduction from action sequences while addressing value overestimation through detached value learning, enabling stable training with any policy architecture.

3 PRELIMINARIES

Problem formulation We consider a Markov Decision Process (MDP) (Sutton & Barto, 2018) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, R, \rho_0, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $R(s, a) : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the reward function, $p(s'|s, a) : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$ is the transition function, ρ_0 is the initial state distribution, and γ is the discount factor. In this paper, we focus on offline reinforcement learning, where we have access only to a static dataset $\mathcal{D} = \{\tau^i\}_{i=0}^N$ containing N trajectories of fixed length H , where each trajectory $\tau^i = (s_0, a_0, r_0, \dots, s_H, a_H, r_H)$ represents a sequence of states, actions, and rewards. The dataset is collected using a data collection policy $\pi_{\mathcal{D}} : \mathcal{S} \rightarrow \Delta(\mathcal{A})$, which may be unknown or suboptimal. Unlike online RL, we cannot interact with the environment during training. The objective is to learn a policy $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ that maximizes the expected sum of discounted rewards $\mathbb{E}_{\rho_0, \pi, p} [\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)]$ using only this fixed dataset.

Options framework To formalize the idea for flexible temporal abstractions in RL and MDP, a Markovian option $\omega \in \Omega$ is defined as a triplet $(\mathcal{I}_{\omega}, \pi_{\omega}, \beta_{\omega})$. $\mathcal{I}_{\omega} \subseteq \mathcal{S}$ is the initiation set, π_{ω} is an *intra-option* policy, and $\beta_{\omega} : \mathcal{S} \rightarrow [0, 1]$ is the termination function. For any MDP $\mathcal{M}$ and any Markovian option $\omega_{\mathcal{M}}$ defined on $\mathcal{M}$, a decision process that follows only the option can be configured as an Semi-Markovian Decision Process (SMDP), which guarantees the existence of a set of optimal policies, denoted as Π_{ω}^* . For more detailed explanations and proofs, please refer to Sutton et al. (1999).

Implicit Q Learning (IQL) (Kostrikov et al., 2022) Instead of regularizing the critic with the actor output, IQL approximates the optimal critic to be maximized only in the region of action distributions present in the offline dataset with an in-sample expectile regression. Given a parameterized critic $Q(s_t, a_t; \theta)$, target critic $Q(s_t, a_t; \bar{\theta})$, and value network $V(s_t; \psi)$, the objective for

value learning is defined as:

$$\begin{aligned}\mathcal{L}_V(\psi) &= \mathbb{E}_{(s_t, a_t) \sim \mathcal{D}} [L_2^\tau(\bar{Q}(s_t, a_t; \bar{\theta}) - V(s_t; \psi))] \\ \mathcal{L}_Q(\theta) &= \mathbb{E}_{(s_t, a_t) \sim \mathcal{D}} [(R(s, a) + \gamma V(s_{t+1}; \psi) - Q(s_t, a_t; \theta))^2]\end{aligned}$$

where $L_2^\tau(u) = |\tau - \mathbb{1}(u < 0)|u^2$ is the expectile loss with expectile parameter $\tau \in [0, 1]$. By using $\tau > 0.5$, Equation 3 penalizes the overestimated value in out-of-distribution actions, letting V and Q to be only approximated in the region of in-distribution actions.

4 METHOD

We propose DETACHED value learning with Action Sequence (DEAS), an offline RL method that models action sequences for scalable learning. Our approach consists of two key components: (1) a critic function $\bar{Q}(s_t, o_t; \bar{\theta})$ that estimates expected returns for the option (consisting of H -step action sequence) $o_t := a_{t:t+H-1}$ from state s_t under the data collection policy $\pi_{\mathcal{D}}$, and (2) a flexible policy update mechanism applicable to any policy $\pi(a_{t:t+H-1}; s_t, \phi)$ that outputs H -step action sequences. Section 4.1 describes how we integrate action sequences into SMDP Q-learning for horizon reduction, while Section 4.2 introduces how DEAS enables stable training through detached value learning, distributional RL, and dual discount factors. We provide pseudocode in Algorithm 1 and implementation details in Appendix A.

4.1 OPTIONS FRAMEWORK FOR ACTION SEQUENCE RL

Complex tasks require coordinated action sequences where each action’s effectiveness depends on its context within the sequence. For instance, in OGBench puzzle or cube tasks, success depends on planning through multiple intermediate steps and maintaining consistent actions over extended periods. These temporal dependencies and hidden sub-tasks are not captured by current state representations, making it challenging for agents to learn effective policies. The challenge becomes even greater in goal-free settings, where agents must discover these sequential patterns from offline data without explicit goal instructions.

To address these challenges, we propose modeling consecutive action sequences as single decision units within the options framework. We treat each H -step action sequence $o_t := a_{t:t+H-1} = \{a_t, a_{t+1}, \dots, a_{t+H-1}\}$ as an option, which naturally induces a SMDP (Bradtke & Duff, 1994; Feinberg, 1994; Sutton et al., 1999; Baykal-Gürsoy & Gürsoy, 2010) that guarantees the existence of an optimal policy. Specifically, the option ω^* is defined as:

$$\begin{aligned}\omega^* &= (\mathcal{I}_{\omega^*}, \pi_{\omega^*}, \beta_{\omega^*}) = (\mathcal{S}, \pi(o_t | s_t), \beta^*(s_t, k)) \\ \beta^*(s_t, k) &= \begin{cases} 1 & \text{if } k = H \\ 0 & \text{otherwise} \end{cases}\end{aligned}$$

where k denotes the number of steps executed within the current option. This leads to a Q-learning update rule that extends standard Q-learning (Bradtke & Duff, 1994):

$$Q(s_t, o_t; \theta) \leftarrow \sum_{k=0}^{H-1} \gamma_1^k R(s_t, a_{t+k}) + \gamma_2^H \max_{o' \in \mathcal{O}} Q(s_{t+H}, o'; \theta)$$

where γ_1 and γ_2 are discount factors for intra-option and inter-option transitions, respectively. This formulation aggregates rewards over H steps and propagates value estimates across temporally extended transitions, achieving horizon reduction similar to n -step TD learning (Park et al., 2025b).

4.2 DEAS: DETACHED VALUE LEARNING WITH ACTION SEQUENCE

Detached value learning for handling action sequence Action sequences introduce challenges for value function approximation, as the expanded action space makes it harder for the critic to estimate Q-values accurately. Meanwhile, the actor can exploit regions where the critic makes prediction errors, leading to value overestimation and unstable learning (Seo & Abbeel, 2025). To address this, we adopt detached value learning (Kostrikov et al., 2022; Xu et al., 2023; Garg et al., 2023) that

Algorithm 1 DEAS

Required: Offline dataset $\mathcal{D}$, Support range for return $\mathbf{v}_{\min}, \mathbf{v}_{\max}$, number of bins m , discount factor γ_1, γ_2
Initialize parameters $\psi, \theta, \bar{\theta}, \phi$
while not converged **do**
 Sample batch $\{(s_t, a_{t:t+H-1}, R_{t:t+H-1}, s_{t+H})\}$ from $\mathcal{D}$
 Compute the discounted return of intra option as $\hat{R}_{t:t+H} = \sum_{k=0}^{H-1} \gamma_1^k R(s_t, a_{t+k})$
 Compute $\bar{Q}(s, o; \bar{\theta})$ and $V(s; \psi)$ using equation (1)
 ▷ Update V Network
 Update $V(s; \psi)$ to minimize Equation (2) with $\bar{Q}(s, o; \bar{\theta})$ and $V(s; \psi)$
 ▷ Update Q Network
 Update $Q(s, o; \theta)$ to minimize Equation (3)
 ▷ Update Actor Network
 Update $\pi(s; \phi)$ with any type of policy extraction algorithms (e.g., BoN, DPG, AWR, etc.)
 Update $\bar{\theta} = (1 - \beta) \cdot \bar{\theta} + \beta \cdot \theta$
return $\pi(s)$

decouples actor and critic training, introducing a critic $Q(s_t, o_t; \theta)$ and a value $V(s_t; \psi)$ networks with the following losses following IQL (Kostrikov et al., 2022):

$$\begin{aligned}\mathcal{L}_V(\psi) &= \mathbb{E}_{(s_t, o_t) \sim \mathcal{D}} [L_2^\tau(\bar{Q}(s_t, o_t; \bar{\theta}) - V(s_t; \psi))] \\ \mathcal{L}_Q(\theta) &= \mathbb{E}_{(s_t, o_t) \sim \mathcal{D}} \left[(\hat{R}_{t:t+H-1} + \gamma_2^H V(s_{t+H}; \psi) - Q(s_t, o_t; \theta))^2 \right],\end{aligned}$$

where $\hat{R}_{t:t+H-1} = \sum_{k=0}^{H-1} \gamma_1^k R(s_t, a_{t+k})$ is the discounted return for the action sequence. This approach steers the critic toward high-return actions in the offline dataset without the potential of exploiting critic approximation errors, preventing value overestimation and enabling stable value learning even with longer action sequences.

Distributional RL for enhanced stability Even with detached value learning, the cumulative reward term $\hat{R}_{t:t+H-1}$ could introduce significant variance when H is large. To enhance stability, we extend our framework with distributional RL (Bellemare et al., 2017; Farebrother et al., 2024), modeling both critic and value networks as categorical distributions over fixed support $[\mathbf{v}_{\min}, \mathbf{v}_{\max}]$ discretized into m bins:

$$Q(s, o; \theta) = \mathbb{E}[Z(s, o; \theta)] \quad Z(s, o; \theta) = \sum_{i=1}^m \hat{p}_i(s, o; \theta) \cdot \delta_{z_i} \quad \hat{p}_i(s, o; \theta) = \frac{e^{l_i(s, o; \theta)}}{\sum_{i=1}^m e^{l_i(s, o; \theta)}}, \quad (1)$$

where $V(s; \psi)$ is computed similarly, by conditioning only on the state s . To address scale differences between regression and classification objectives, we maintain IQL’s weighting scheme but replace regression with classification-based learning:

$$\begin{aligned}\mathcal{L}_V(\psi) &= \mathbb{E}_{(s_t, o_t) \sim \mathcal{D}} \left[\alpha_t \cdot \sum_{i=1}^m \hat{p}_i(s_t; \psi) \log \hat{p}_i(s_t, o_t; \bar{\theta}) \right] \\ \alpha_t &= \begin{cases} \tau & \text{if } \bar{Q}(s_t, o_t; \bar{\theta}) \geq V(s_t; \psi) \\ 1 - \tau & \text{otherwise,} \end{cases}\end{aligned} \quad (2)$$

$$\mathcal{L}_Q(\theta) = \mathbb{E}_{(s_t, a_{t:t+H-1}, s_{t+H}) \sim \mathcal{D}} \left[\sum_{i=1}^m p_i(s_t; \psi) \log \hat{p}_i(s_t, a_{t:t+H-1}; \hat{\theta}) \right]. \quad (3)$$

For target probabilities p_i , we adopt the truncated normal distribution with mean as Bellman target $(\hat{\mathcal{T}}\mathcal{V})(s, a_{t:t+H-1}) = \sum_{k=0}^{H-1} \gamma_1^k r_{t+k} + \gamma_2^H V(s_{t+H}; \psi)$ and standard deviation $\sigma = 0.75 \cdot (v_{\max} - v_{\min})/m$, inspired by Farebrother et al. (2024).

Dual discount factors To further enhance stability and expressiveness in value estimation, we employ two separate discount factors: γ_1 for intra-option (within action sequence) rewards and γ_2 for inter-option (across action sequences) rewards. This dual-discounting scheme enables the value



Figure 2: **Simulation task examples.** We study DEAS on 30 different tasks from OGBench (Park et al., 2025a) and 4 challenging manipulation tasks from RoboCasa Kitchen (Nasiriany et al., 2024).

function to appropriately weigh immediate and future returns, mitigating issues such as value explosion or collapse that can arise from improper scaling of returns. In our experiments, we observe that decreasing the intra-option discount factor γ_1 and increasing the inter-option discount factor γ_2 leads to more stable training, and is critical for stable training, especially when the action sequence becomes longer (see Section 5.3 for the supporting results).

Compatible policy methods For obtaining final policy $\pi(s; \phi)$, our framework is compatible with a variety of policy extraction strategies (Park et al., 2024), including weighted behavior cloning (Peng et al., 2019), deterministic policy gradient (DPG) (Fujimoto & Gu, 2021), best-of-N sampling (Chen et al., 2023), and flow-matching approaches (Park et al., 2025c). Since value function training does not require querying the policy, it can be performed independently, and the policy can be updated separately. To demonstrate this, we illustrate the effectiveness of our method using various policy extraction methods in our experiments.

5 EXPERIMENTS

We first validate the effectiveness of DEAS through extensive experiments on various complex tasks in OGBench (Park et al., 2025a). Additionally, to prove that DEAS can be naturally plugged into large-scale VLAs for practical applications, we evaluate DEAS by fine-tuning GR00T N1.5 (NVIDIA, 2025) using offline RL methods on 4 hard tasks from RoboCasa Kitchen (Nasiriany et al., 2024) and also conduct real-world experiments with Franka Emika Research 3 Robot Arm. See Figure 2 and Figure 4 for task examples used in our experiments.

5.1 OGBENCH EXPERIMENTS

Setup We evaluate on 6 manipulation environments from OGBench (Park et al., 2025a), each with 5 subtasks. We use datasets ranging from 1M to 100M transitions based on task difficulty. While OGBench is originally designed for offline goal-conditioned RL, we use its single-task variants (‘-singletask’) for reward-maximizing offline RL. For fair comparison, all methods use identical MLP architectures for actor networks and adopt the same policy extraction approach as FQL (Park et al., 2025c), except for CQN-AS, which uses value function networks as the actor itself through discretization. Action sequence length H is set to 8 for scene and puzzle tasks, and 4 for cube tasks, with $n = H$ is used for n -step FQL. More details about the experimental setup can be found in Appendix A.1.

Baselines We compare against FQL (Park et al., 2025c), a state-of-the-art offline RL method using one-step distillation between flow matching models with different denoising steps, and n -step FQL (Sutton & Barto, 2018), which extends FQL with n -step TD updates for horizon reduction (Park et al., 2025b). While increasing n increases bias in standard offline RL, DEAS explicitly models action sequences while maintaining horizon reduction benefits. We also consider Q-Chunking (QC) (Li et al., 2025b), which uses action chunking for actor-critic training while keeping the interaction between actor and critic, while DEAS uses detached value learning. For fair comparison with ours, we extensively tune QC-FQL hyperparameters to achieve higher performance than the original paper. Lastly, CQN-AS (Seo & Abbeel, 2025), a value-based RL method with action sequence utilizing multi-level critics with iterative discretization, is included as a baseline.

Quantitative results As shown in Table 1, DEAS consistently achieves the best performance across all 6 task categories with various dataset sizes. Comparing FQL and N -step FQL, we observe that simply increasing the n -step mostly leads to performance degradation due to bias in standard offline RL, while our detached value learning approach enables stable training with action sequences. Notably, DEAS matches or outperforms QC-FQL across all tasks, demonstrating

Table 1: **Offline RL results** in 6 task categories from OGBench (Park et al., 2025a). We report the success rate (%) and 95% stratified bootstrap confidence interval over 4 runs. **Bold** indicates the values at or above 95% of the best performance. Please refer to Table 8 for the full results.

Task Category	#Data	FQL	N-step FQL	QC-FQL	CQN-AS	DEAS
scene-play-singletask (5 tasks)	1M	50 $\pm$ 3	36 $\pm$ 2	73 $\pm$ 2	1 $\pm$ 1	76 $\pm$ 2
cube-double-play-singletask (5 tasks)		14 $\pm$ 2	4 $\pm$ 2	41 $\pm$ 3	2 $\pm$ 1	48 $\pm$ 2
puzzle-3x3-play-singletask (5 tasks)		44 $\pm$ 3	36 $\pm$ 3	62 $\pm$ 7	0 $\pm$ 0	91 $\pm$ 3
cube-triple-play-singletask (5 tasks)	10M	10 $\pm$ 3	23 $\pm$ 2	83 $\pm$ 4	0 $\pm$ 0	82 $\pm$ 5
puzzle-4x4-play-singletask (5 tasks)		32 $\pm$ 4	19 $\pm$ 5	69 $\pm$ 8	0 $\pm$ 0	82 $\pm$ 6
cube-quadruple-play-singletask (5 tasks)	100M	17 $\pm$ 8	36 $\pm$ 10	45 $\pm$ 7	0 $\pm$ 0	64 $\pm$ 8

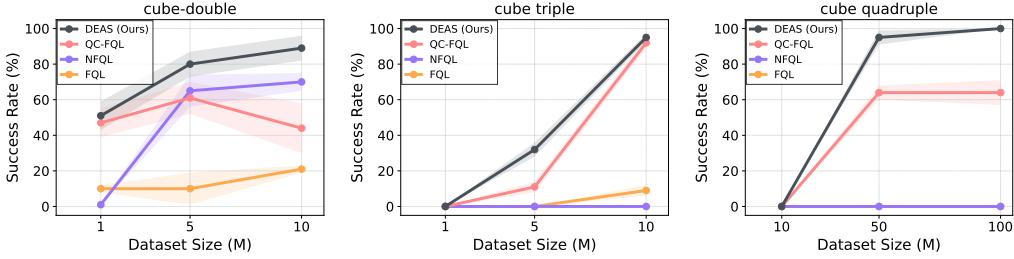


Figure 3: Agent performance across varying dataset sizes on three representative OGBench (Park et al., 2025a) tasks, evaluated by success rate (%). Solid lines indicate the mean, while shaded areas denote the stratified bootstrap confidence intervals over 4 independent runs.

the effectiveness of our stable value learning in addressing offline RL instability. The method shows particularly strong performance on tasks requiring long-horizon reasoning like puzzle and the most challenging tasks (i.e., cube – quadruple), where the benefits of using action sequences are most pronounced. CQN-AS shows significantly lower performance, likely due to its direct application of strong BC regularization on the value function in the presence of predominantly suboptimal data, along with cumulative errors from iterative discretizations that reduce action precision.

Scaling analysis To further validate the scalability of DEAS, we conduct a scaling analysis on three representative OGBench tasks with varying dataset sizes. As shown in Figure 3, DEAS consistently outperforms all baselines across all dataset sizes, achieving the highest success rates in every environment. The method demonstrates robust scaling across different dataset sizes, maintaining consistent performance gains even with larger datasets. This superior performance validates our approach of explicitly modeling action sequences while effectively leveraging suboptimal data through our detached value learning and stable multi-step training.

5.2 VLA EXPERIMENTS

To validate the practical applicability of DEAS, we demonstrate its effectiveness with large-scale VLAs (Black et al., 2025; Bjorck et al., 2025; NVIDIA, 2025). These models, trained on internet-scale diverse datasets with billion-scale parameters, predict much longer action sequences and are widely used in robotics applications. However, deploying these models typically requires fine-tuning on task-specific data, which often necessitates collecting expensive expert demonstrations. We design our experiments to validate whether DEAS can improve VLA performance by effectively utilizing suboptimal demonstrations alongside limited expert data, potentially reducing the required amount of costly expert demonstrations. See Appendix A.2 for more details.

5.2.1 ROBOCASA KITCHEN EXPERIMENTS

Setup We employ GR00T N1.5 (NVIDIA, 2025) as the backbone VLA. First, we fine-tune the VLA using 100 expert demonstrations from all 24 RoboCasa Kitchen tasks to verify that we achieve performance similar to the original GR00T N1 (Bjorck et al., 2025). From these tasks, we select 4 tasks with the lowest success rates in their respective categories for our offline IL/RL experiments. We then collect 300 rollouts for each task from the resulting policy and apply various offline IL/RL methods. For RL methods, we fine-tune the base policy using behavior cloning on both expert demonstrations and the rollout dataset and use the model as an actor for training critic functions when necessary. For policy extraction, we adopt best-of-N sampling (Chen et al., 2023; Nakamoto

Table 2: **RoboCasa Kitchen evaluation results.** We fine-tune GR00T N1.5 (NVIDIA, 2025) on 24 RoboCasa Kitchen tasks using 100 expert demonstrations per task. For 4 selected tasks, we collect 300 rollouts and apply offline IL/RL algorithms. Success rates (%) on 50 episodes, aggregated with 3 seeds. PnP2M denotes ‘PnPCounterToMicrowave’ and PnP2C denotes ‘PnPMicrowaveToCounter’. **Bold** and underline indicate best and runner-up results, respectively.

				† Reproduced performance	
Models	CoffeeSetupMug	PnPC2M	PnPM2C	TurnOffStove	Avg.
<i>Base models</i>					
GR00T N1*	2.0	0.0	0.0	15.7	4.4
GR00T N1.5†	4.7	21.3	7.3	14.7	12.0
<i>Imitation learning</i>					
+ Filtered BC	14.7	25.3	<u>14.7</u>	19.3	18.5
<i>Offline RL</i>					
+ IQL	<u>23.3</u>	<u>30.0</u>	<u>14.7</u>	12.7	<u>20.2</u>
+ QC	16.0	28.7	<u>14.7</u>	10.7	17.5
+ DEAS (Ours)	28.7	36.0	18.0	<u>18.0</u>	25.2

et al., 2024), where we sample multiple outputs from the policy and select the action sequence with the highest Q-value. We set $H = 16$ for all methods, matching GR00T N1.5’s action chunk size.

Baselines We compare against several baselines across both imitation learning and reinforcement learning paradigms. For imitation learning, we consider **Filtered BC**, which fine-tunes the base policy using both expert demonstrations and successful episodes from the rollout data (Oh et al., 2018). For reinforcement learning, we evaluate **IQL**, a value-based method that operates on single actions without requiring policy outputs. For determining action sequence in IQL, we use the very first action in the sequence for value estimation. Lastly, we consider **QC**, which employs action chunking for critic training but relies on predicted action sequences from VLA for the critic update.

Results As shown in Table 2, DEAS achieves the highest success rates in 3 out of 4 tasks, with the remaining task also showing improved performance compared to the base model. While filtered BC improves performance with simple approaches, but our approach exhibits additional performance gains by effectively utilizing suboptimal data. While single-step IQL also demonstrates effectiveness, it shows smaller performance gains across all tasks compared to our approach, due to its lack of understanding of action sequences. QC shows limited improvement compared to BC-based approaches, highlighting the advantage of our detached value learning with action sequences.

5.2.2 REAL-WORLD EXPERIMENTS

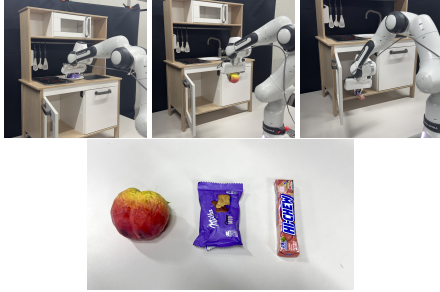
Setup We further investigate the effectiveness of DEAS in real-world tasks using Franka Emika Research 3 Robot Arm. Inspired by RoboCasa Kitchen, we design pick-and-place tasks from the countertop to the bottom cabinet, with three different objects: peach, milka, and hichew (see Figure 4). For each task, we collect 5 demonstrations, fine-tune GR00T N1.5, collect 25 rollouts, and apply various offline IL/RL methods. We evaluate using 20 rollouts per task from 5 different initial points and use the same baselines as in the RoboCasa Kitchen experiments. Success rates are calculated based on partial success scoring (0-1 scale) that considers subtask completion, with detailed evaluation methodology provided in Section A.2.2.

Results In Table 3, DEAS achieves the highest success rates across all three pick-and-place tasks compared to baselines. The method shows consistent improvements, particularly on challenging objects like milka (a deformable object) where other approaches struggle. Notably, QC shows degraded performance compared to the base model, likely due to its instability when using action sequences with relatively small datasets, while our method shows stable improvement even with limited data. These results demonstrate that our detached value learning approach can be effectively applied to real-world robotic tasks and remains stable regardless of the dataset size.

5.3 ABLATION STUDIES AND ANALYSES

We investigate the effect of hyperparameters and various components of DEAS by running experiments on OGBench puzzle-4x4 task.

Figure 4: **Real-world tasks.** We conduct pick-and-place tasks from the countertop to the bottom cabinet with peach, milka, and hichew.



H	Actor	SR
1	512×4	21 ± 3
2	512×4	25 ± 5
4	512×4	75 ± 8
8	512×4	88 ± 4
16	512×4	51 ± 4
16	1024×4	84 ± 4

(a) Action sequence

Critic	Value	SR
256×4	256×4	69 ± 7
512×4	256×4	88 ± 4
1024×4	256×4	91 ± 4
512×4	512×4	50 ± 4

(b) Critic size

IQL	HLG	SR
✗	✓	75 ± 5
✓	✗	63 ± 6
✓	✓	88 ± 4

(c) Objectives

γ_1	γ_2	SR
0.8	0.999	87 ± 4
0.9	0.999	88 ± 4
0.99	0.999	81 ± 5
0.999	0.999	80 ± 8

(d) γ_1 and γ_2

Table 3: **Real-world evaluation results.** We report the partial success rate (% , over 20 trials per task) on 3 tasks from 5 initial points. **Bold** and underline indicate best and runner-up results, respectively.

Models	peach	milka	hichew	Avg.
<i>Base model</i>				
GR00T N1.5	62.0	<u>45.0</u>	85.0	64.0
<i>Imitation learning</i>				
+ Filtered BC	76.3	25.0	<u>92.5</u>	64.6
<i>Offline RL</i>				
+ IQL	82.5	37.5	78.8	66.3
+ QC	58.8	15.0	45.0	39.6
+ DEAS (Ours)	86.3	53.8	95.0	78.4

Table 4: **Ablation studies.** We investigate the effect of (a) action sequence length H , (b) critic and value model size, (c) training objectives, and (d) separate discount factors γ_1 and γ_2 for intra-option and inter-option rewards. SR denotes success rate (%) and default settings are highlighted in gray. **Bold** indicates values at or above 95% of the best performance.

Effect of action sequence length Table 4a investigates the impact of action sequence length on performance. When using single-step or two-step action ($H = 1, 2$), DEAS fails to achieve meaningful performance, confirming the necessity of action sequences for long-horizon tasks. Performance improves with longer sequences, but when the sequence length becomes longer than 8, it requires proportionally larger actor networks to handle the increased action dimensions, suggesting a trade-off between sequence length and computational efficiency.

Effect of network size Table 4b analyzes the sensitivity to network sizes. For the critic network, we observe that increasing capacity initially improves performance by better approximating the value function. For the value function, we find that the network needs sufficient capacity to capture the complexity of action sequence values, but excessive capacity without proper regularization causes instability in value estimation, leading to performance degradation.

Effect of training objective In Table 4c, we compare different training objectives for value estimation. We found that using only distributional RL (HLG) (Farebrother et al., 2024) or only standard regression (IQL) shows limited performance. However, combining detached value learning with distributional estimation significantly improves results, suggesting both components are crucial for stable training with action sequences.

Effect of dual discount factors Lastly, we examine the effect of dual discount factors on learning dynamics in Table 4d. Proper tuning of γ_1 (the discount factor for action sequences) is essential for performance, as it controls the temporal horizon for value estimation within sequences. In this paper, we use $\gamma_1 = 0.9$ for all experiments.

6 CONCLUSION

We present DEAS, a simple yet effective offline RL method that leverages action sequences for scalable learning in complex tasks. By modeling temporally extended actions through the options framework, DEAS achieves principled horizon reduction via SMDP Q-learning while addressing value overestimation through detached value learning. Our experiments demonstrate consistent improvements over baselines on challenging OGBench tasks and successful application to large-scale VLAs, showing the practical potential for scaling offline RL to real-world scenarios.

REPRODUCIBILITY STATEMENT

We provide full hyperparameter and implementation details in Section 5 and Section A. In addition, to further facilitate the reproduction, we release the open-sourced implementation through the [project website](#).

ACKNOWLEDGMENTS

CY thanks Jaehyun Nam, Juyong Lee, and anonymous reviewers for providing helpful feedback and suggestions for improving our paper. CY also thanks Angeline S. Kim for the assistance in improving the expression and the visualization of the paper.

REFERENCES

- Anurag Ajay, Aviral Kumar, Pulkit Agrawal, Sergey Levine, and Ofir Nachum. {OPAL}: Offline primitive discovery for accelerating offline reinforcement learning. In *International Conference on Learning Representations*, 2021.
- Pierre-Luc Bacon, Jean Harb, and Doina Precup. The option-critic architecture. In *AAAI Conference on Artificial Intelligence*, 2017.
- Melike Baykal-Gürsoy and K Gürsoy. Semi-markov decision processes. *Wiley Encyclopedia of Operations Research and Management Sciences*, 2010.
- Marc G Bellemare, Will Dabney, and Rémi Munos. A distributional perspective on reinforcement learning. In *International Conference on Machine Learning*, 2017.
- Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. In *Robotics: Science and Systems*, 2025.
- Steven Bradtke and Michael Duff. Reinforcement learning methods for continuous-time markov decision problems. In *Conference on Neural Information Processing Systems*, 1994.
- Yevgen Chebotar, Quan Vuong, Karol Hausman, Fei Xia, Yao Lu, Alex Irpan, Aviral Kumar, Tianhe Yu, Alexander Herzog, Karl Pertsch, et al. Q-transformer: Scalable offline reinforcement learning via autoregressive q-functions. In *Conference on Robot Learning*, 2023.
- Huayu Chen, Cheng Lu, Chengyang Ying, Hang Su, and Jun Zhu. Offline reinforcement learning via high-fidelity generative behavior modeling. In *International Conference on Learning Representations*, 2023.
- Yuhui Chen, Shuai Tian, Shugao Liu, Yingting Zhou, Haoran Li, and Dongbin Zhao. Conrft: A reinforced fine-tuning method for vla models via consistency policy. *arXiv preprint arXiv:2502.05450*, 2025a.
- Zengjue Chen, Runliang Niu, He Kong, and Qi Wang. Tgrpo: Fine-tuning vision-language-action model via trajectory-wise group relative policy optimization. *arXiv preprint arXiv:2506.08440*, 2025b.
- Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *International Journal of Robotics Research*, 2023.
- Jesse Farebrother, Jordi Orbay, Quan Vuong, Adrien Ali Taiga, Yevgen Chebotar, Ted Xiao, Alex Irpan, Sergey Levine, Pablo Samuel Castro, Aleksandra Faust, Aviral Kumar, and Rishabh Agarwal. Stop regressing: Training value functions via classification for scalable deep RL. In *International Conference on Machine Learning*, 2024.

- Eugene A Feinberg. Constrained semi-markov decision processes with average rewards. *Zeitschrift für Operations Research*, 1994.
- Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4rl: Datasets for deep data-driven reinforcement learning. *arXiv preprint arXiv:2004.07219*, 2020.
- Scott Fujimoto and Shixiang Shane Gu. A minimalist approach to offline reinforcement learning. In *Conference on Neural Information Processing Systems*, 2021.
- Scott Fujimoto, David Meger, and Doina Precup. Off-policy deep reinforcement learning without exploration. In *International Conference on Machine Learning*, 2019.
- Divyansh Garg, Joey Hejna, Matthieu Geist, and Stefano Ermon. Extreme q-learning: Maxent RL without entropy. In *International Conference on Learning Representations*, 2023.
- Caglar Gulcehre, Ziyu Wang, Alexander Novikov, Thomas Paine, Sergio Gómez, Konrad Zolna, Rishabh Agarwal, Josh S Merel, Daniel J Mankowitz, Cosmin Paduraru, et al. RL unplugged: A suite of benchmarks for offline reinforcement learning. *Conference on Neural Information Processing Systems*, 2020.
- Yanjiang Guo, Jianke Zhang, Xiaoyu Chen, Xiang Ji, Yen-Jen Wang, Yucheng Hu, and Jianyu Chen. Improving vision-language-action model with online reinforcement learning. *arXiv preprint arXiv:2501.16664*, 2025.
- Philippe Hansen-Estruch, Ilya Kostrikov, Michael Janner, Jakub Grudzien Kuba, and Sergey Levine. Idql: Implicit q-learning as an actor-critic method with diffusion policies. *arXiv preprint arXiv:2304.10573*, 2023.
- Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.
- Dongchi Huang, Zhirui Fang, Tianle Zhang, Yihang Li, Lin Zhao, and Chunhe Xia. Co-rft: Efficient fine-tuning of vision-language-action models through chunked offline reinforcement learning. *arXiv preprint arXiv:2508.02219*, 2025.
- Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- Michael J Kearns and Satinder Singh. Bias-variance error bounds for temporal difference updates. In *Conference on Learning Theory*, 2000.
- Diederik P Kingma. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.
- Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit q-learning. In *International Conference on Learning Representations*, 2022.
- Tejas D Kulkarni, Karthik Narasimhan, Ardavan Saeedi, and Josh Tenenbaum. Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation. In *Conference on Neural Information Processing Systems*, 2016.
- Aviral Kumar, Justin Fu, Matthew Soh, George Tucker, and Sergey Levine. Stabilizing off-policy q-learning via bootstrapping error reduction. In *Conference on Neural Information Processing Systems*, 2019.
- Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative q-learning for offline reinforcement learning. In *Conference on Neural Information Processing Systems*, 2020.

- Aviral Kumar, Rishabh Agarwal, Xinyang Geng, George Tucker, and Sergey Levine. Offline q-learning on diverse multi-task data both scales and generalizes. In *International Conference on Learning Representations*, 2023a.
- Aviral Kumar, Anikait Singh, Frederik Ebert, Mitsuhiko Nakamoto, Yanlai Yang, Chelsea Finn, and Sergey Levine. Pre-training for robots: Offline rl enables learning new tasks from a handful of trials. In *Robotics: Science and Systems*, 2023b.
- Sascha Lange, Thomas Gabel, and Martin Riedmiller. Batch reinforcement learning. In *Reinforcement learning: State-of-the-art*, 2012.
- Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*, 2020.
- Ge Li, Dong Tian, Hongyi Zhou, Xinkai Jiang, Rudolf Lioutikov, and Gerhard Neumann. Top-erl: Transformer-based off-policy episodic reinforcement learning. *arXiv preprint arXiv:2410.09536*, 2024.
- Haozhan Li, Yuxin Zuo, Jiale Yu, Yuhao Zhang, Zhaohui Yang, Kaiyan Zhang, Xuekai Zhu, Yuchen Zhang, Tianxing Chen, Ganqu Cui, et al. Simplevla-rl: Scaling vla training via reinforcement learning. *arXiv preprint arXiv:2509.09674*, 2025a.
- Qiyang Li, Zhiyuan Zhou, and Sergey Levine. Reinforcement learning with action chunking. *arXiv preprint arXiv:2507.07969*, 2025b.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- Ajay Mandlekar, Danfei Xu, Josiah Wong, Soroush Nasiriany, Chen Wang, Rohun Kulkarni, Li Fei-Fei, Silvio Savarese, Yuke Zhu, and Roberto Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. *arXiv preprint arXiv:2108.03298*, 2021.
- Ajay Mandlekar, Soroush Nasiriany, Bowen Wen, Ireteayo Akinola, Yashraj Narang, Linxi Fan, Yuke Zhu, and Dieter Fox. Mimicgen: A data generation system for scalable robot learning using human demonstrations. In *Conference on Robot Learning*, 2023.
- Ofir Nachum, Shixiang Shane Gu, Honglak Lee, and Sergey Levine. Data-efficient hierarchical reinforcement learning. In *Conference on Neural Information Processing Systems*, 2018.
- Ashvin Nair, Abhishek Gupta, Murtaza Dalal, and Sergey Levine. Awac: Accelerating online reinforcement learning with offline datasets. *arXiv preprint arXiv:2006.09359*, 2020.
- Mitsuhiko Nakamoto, Simon Zhai, Anikait Singh, Max Sobol Mark, Yi Ma, Chelsea Finn, Aviral Kumar, and Sergey Levine. Cal-ql: Calibrated offline rl pre-training for efficient online fine-tuning. In *Conference on Neural Information Processing Systems*, 2023.
- Mitsuhiko Nakamoto, Oier Mees, Aviral Kumar, and Sergey Levine. Steering your generalists: Improving robotic foundation models via value guidance. In *Conference on Robot Learning*, 2024.
- Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. Robocasa: Large-scale simulation of everyday tasks for generalist robots. In *Robotics: Science and Systems*, 2024.
- Michal Nauman, Mateusz Ostaszewski, Krzysztof Jankowski, Piotr Miłoś, and Marek Cygan. Bigger, regularized, optimistic: scaling for compute and sample efficient continuous control. In *Conference on Neural Information Processing Systems*, 2024.
- NVIDIA. Gr00t n1.5: An improved open foundation model for generalist humanoid robots. https://research.nvidia.com/labs/gear/gr00t-n1_5/, June 2025. Accessed: 2025-09-09.
- Junhyuk Oh, Yijie Guo, Satinder Singh, and Honglak Lee. Self-imitation learning. In *International Conference on Machine Learning*, 2018.

- Seohong Park, Kevin Frans, Sergey Levine, and Aviral Kumar. Is value learning really the main bottleneck in offline rl? In *Conference on Neural Information Processing Systems*, 2024.
- Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. OGBench: Benchmarking offline goal-conditioned RL. In *International Conference on Learning Representations*, 2025a.
- Seohong Park, Kevin Frans, Deepinder Mann, Benjamin Eysenbach, Aviral Kumar, and Sergey Levine. Horizon reduction makes rl scalable. In *Conference on Neural Information Processing Systems*, 2025b.
- Seohong Park, Qiyang Li, and Sergey Levine. Flow q-learning. In *International Conference on Machine Learning*, 2025c.
- Xue Bin Peng, Aviral Kumar, Grace Zhang, and Sergey Levine. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning. *arXiv preprint arXiv:1910.00177*, 2019.
- Dean A Pomerleau. Alvin: An autonomous land vehicle in a neural network. In *Conference on Neural Information Processing Systems*, 1988.
- Younggyo Seo and Pieter Abbeel. Coarse-to-fine q-network with action sequence for data-efficient robot learning. In *Conference on Neural Information Processing Systems*, 2025.
- Jost Tobias Springenberg, Abbas Abdolmaleki, Jingwei Zhang, Oliver Groth, Michael Bloesch, Thomas Lampe, Philemon Brakel, Sarah Maria Elisabeth Bechtle, Steven Kapturowski, Roland Hafner, Nicolas Heess, and Martin Riedmiller. Offline actor-critic reinforcement learning scales to large models. In *International Conference on Machine Learning*, 2024.
- Martin Stolle and Doina Precup. Learning options in reinforcement learning. In *International Symposium on abstraction, reformulation, and approximation*, 2002.
- Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
- Richard S Sutton, Doina Precup, and Satinder Singh. Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. *Artificial intelligence*, 1999.
- Shuhan Tan, Kairan Dou, Yue Zhao, and Philipp Krähenbühl. Ript-vla: Interactive post-training for vision-language-action models. *arXiv preprint arXiv:2505.17016*, 2025.
- Denis Tarasov, Vladislav Kurenkov, Alexander Nikulin, and Sergey Kolesnikov. Revisiting the minimalist approach to offline reinforcement learning. In *Conference on Neural Information Processing Systems*, 2023.
- Dong Tian, Ge Li, Hongyi Zhou, Onur Celik, and Gerhard Neumann. Chunking the critic: A transformer-based soft actor-critic with n-step returns. *arXiv preprint arXiv:2503.03660*, 2025.
- John Tsitsiklis and Benjamin Van Roy. Analysis of temporal-difference learning with function approximation. In *Conference on Neural Information Processing Systems*, 1996.
- Alexander Sasha Vezhnevets, Simon Osindero, Tom Schaul, Nicolas Heess, Max Jaderberg, David Silver, and Koray Kavukcuoglu. Feudal networks for hierarchical reinforcement learning. In *International Conference on Machine Learning*, 2017.
- Zhendong Wang, Jonathan J Hunt, and Mingyuan Zhou. Diffusion policies as an expressive policy class for offline reinforcement learning. In *International Conference on Learning Representations*, 2023.
- Ziyu Wang, Alexander Novikov, Konrad Zolna, Josh S Merel, Jost Tobias Springenberg, Scott E Reed, Bobak Shahriari, Noah Siegel, Caglar Gulcehre, Nicolas Heess, et al. Critic regularized regression. In *Conference on Neural Information Processing Systems*, 2020.
- Haoran Xu, Li Jiang, Jianxiong Li, Zhuoran Yang, Zhaoran Wang, Victor Wai Kin Chan, and Xianyu Zhan. Offline RL with no OOD actions: In-sample learning via implicit value regularization. In *International Conference on Learning Representations*, 2023.

Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. In *Conference on Robot Learning*, 2020.

Hongyin Zhang, Zifeng Zhuang, Han Zhao, Pengxiang Ding, Hongchao Lu, and Donglin Wang. Reinbot: Amplifying robot visual-language manipulation with reinforcement learning. *arXiv preprint arXiv:2505.07395*, 2025.

Zijian Zhang, Kaiyuan Zheng, Zhaorun Chen, Joel Jang, Yi Li, Siwei Han, Chaoqi Wang, Mingyu Ding, Dieter Fox, and Huaxiu Yao. Grape: Generalizing robot policy via preference alignment. *arXiv preprint arXiv:2411.19309*, 2024.

Tony Z Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems*, 2023.

A IMPLEMENTATION AND TRAINING DETAILS

A.1 OGBENCH EXPERIMENTS

Tasks We evaluate our method on 6 UR5 Robot Arm manipulation environments from OGBench (Park et al., 2025a), each with 5 subtasks. All tasks are state-based, and goal-free setup. For each task, the observation space consists of the proprioceptive state of the UR5 Robot Arm, and low-dim state vector informing the target object state and position. The action space consists of the cartesian position of UR5 robot arm, gripper yaw, and gripper open/close. For substituting goal-conditioned environment to standard function, we use the simple semi-sparse reward function, which is defined as the negative number of uncompleted subtasks in the current state, following Park et al. (2025a). For all tasks, the maximum episode length is set to 1000.

Implementation details We implement our method on top of the open-source implementation of FQL (Park et al., 2025c) ¹. Unless otherwise mentioned, we largely follow the training/evaluation setup and network architecture from Park et al. (2025c) and Park et al. (2025b). For training value network, we use the smaller size network compared to critic network for all experiments, which shows the best performance, and we use the doubled size of network for the critic network. For cube experiments, we use BRO (Nauman et al., 2024) for additional regularization between relatively small range of returns in value function training. For selecting $\mathbf{v}_{\min}$ and $\mathbf{v}_{\max}$ for distributional RL, we use two procedures: 1) *data-centric*: compute return distribution from the dataset and select 1% and 99% quantiles with 20% padding, and 2) *universal*: compute theoretical bounds using reward range $[r_{\min}, r_{\max}]$, horizon L , option length H , and discount factors γ_1, γ_2 . For SMDP with $K = L/H$ options, the theoretical bounds are:

$$\mathbf{v}_{\min} = r_{\min} \frac{1 - \gamma_2^H}{1 - \gamma_2} \frac{1 - \gamma_1^K}{1 - \gamma_1} \quad (4)$$

$$\mathbf{v}_{\max} = r_{\max} \frac{1 - \gamma_2^H}{1 - \gamma_2} \frac{1 - \gamma_1^K}{1 - \gamma_1} \quad (5)$$

where γ_1 and γ_2 denote option-level and action-level discount factors, respectively.

Training and evaluation For the training dataset, we use the open-sourced 1M/100M play dataset released by Park et al. (2025a) ², where the dataset is collected by open-loop, non-Markovian scripted policies with temporally correlated noise. As 100M dataset consists of 100 separate files with 1M transitions for each, we use the first 10 files sorted by name for 10M dataset. We train our method and baselines for 1M (1M data) / 2.5M (10M/100M data) gradient steps. For selecting BC coefficient α for policy extraction, we first normalize the Q loss as in Fujimoto & Gu (2021) and sweep the value from $\{0.1, 0.3, 1, 3, 10\}$ and choose the best one for each task and baseline, except cube – double, where we follow the hyperparameter used in Li et al. (2025b). For evaluation, we report the average success rates across the last three evaluation epochs (800K, 900K, 1M for 1M dataset, 2.3M, 2.4M, 2.5M for 10M/100M dataset) following Park et al. (2025c) and Park et al. (2025b). For checking additional hyperparameters used in our experiments, please refer to Section A.3.

Baselines For reporting results from FQL and n -step FQL, we use the implementation from Park et al. (2025c). For Q-Chunking, we re-implement the code from Li et al. (2025b) ³ in our codebase. We found that simply increasing discount factor γ leads to significant performance improvement for Q-Chunking, so we use the discount factor to be same with γ_2 for value function training. For implementing CQN-AS, we use the original implementation released by the authors from Seo & Abbeel (2025) ⁴ and integrate OGBench related codes on top of the codebase. Originally, CQN-AS is designed to apply auxiliary BC loss only on expert demonstrations, but considering the dataset distribution of OGBench tasks with nearly no success rollouts, we modify the BC loss on the suboptimal data as well (Fujimoto & Gu, 2021; Park et al., 2025c; 2024), where no significant difference

¹<https://github.com/seohongpark/fql>

²<https://github.com/seohongpark/ogbench>

³<https://github.com/ColinQiyangLi/qc>

⁴<https://github.com/younggyoseo/CQN-AS>

with the original implementation. As the reward scale for OGBench is highly different according to the domain, we normalize the reward scale to be in $[-1, 0]$, and use $\mathbf{v}_{\min}$ and $\mathbf{v}_{\max}$ as -200 and 0 , respectively. For levels and bins, we use 5 (level) and 9 (bins) for all experiments.

Computing hardware For all OGBench experiments, we use a single NVIDIA RTX 3090 GPU with 24GB VRAM and it takes about 2 hours for training the small model (used for 1M dataset) and about 8 hours for training the large model (used for 10M/100M dataset).

A.2 VLA EXPERIMENTS

Computing hardware For all VLA experiments, we use NVIDIA A100 80GB GPUs. Fine-tuning GR00T N1.5 takes about 4 hours for 100 expert demonstrations and successful rollouts. For training DEAS and baselines, it takes about 10 hours with the same data, as we use a larger batch size.

VLA fine-tuning We implement our method and baselines on top of the open-source implementation of GR00T N1.5 (NVIDIA, 2025)⁵. As our code is based on an earlier version of GR00T N1.5, we conduct experiments without introducing future tokens to the action expert modules. For fine-tuning GR00T N1.5, we use a batch size of 32 and train for 30K (RoboCasa Kitchen) / 10K (Real Robot) steps using AdamW (Loshchilov & Hutter, 2019) optimizer with learning rate 1×10^{-4} and cosine annealing schedule.

A.2.1 ROBOCASA KITCHEN EXPERIMENTS

Task RoboCasa Kitchen (Nasiriany et al., 2024) is a simulation environment with a mobile manipulator attached to a Franka Panda robot arm in household kitchen environments. Among 24 atomic tasks provided by the environment, we select 4 challenging tasks (CoffeeSetupMug, PnPmicrowaveToCounter, PnPmicrowaveToMicrowave, PnPmicrowaveToStove) that require relatively long-horizon and delicate manipulation with small grasping part, which is demonstrated by the low success rate of the base model. For perception, camera images from 3 different view-points (left front, right front, wrist), proprioceptive states including position/velocities of joint/base, and natural language instructions, are provided. For reward function, we use the pre-defined success detector in the environment, and use the sparse reward function where the reward is 1 if the task is completed, and 0 otherwise.

Implementation details As an input for the value function, we first use the proprioceptive states from the robot, including joint position/angle, base position/orientation for the mobile manipulator. To provide information on target objects to the value function, we utilize the encoded representation of three different camera views and task instructions from the VLM backbone. For the value/critic network architecture, we use the same hyperparameters as those used for the 100M dataset experiments. For optimizing value and critic function, we use the expectile parameter τ as 0.7, and use $\gamma_1 = 0.9$, $\gamma_2 = 0.99$, *universal* support type for distributional RL, for all experiments. For selecting action candidates with the value function, we first sample $N = 10$ candidates from the policy. For selecting final actions, we try either 1) greedy sampling with highest Q-value or 2) inspired by Nakamoto et al. (2024), sampling the action from a categorical distribution obtained by temperature controlled softmax over Q-values: $a_t \sim \text{Softmax}(\frac{Q(s_t, a_1)}{\beta}, \dots, \frac{Q(s_t, a_N)}{\beta})$ with temperature $\beta = 1$ and report the best result for each task.

Training and evaluation For expert demonstrations, we randomly sample 100 expert demonstrations using the publicly available dataset generated by MimicGen (Mandlekar et al., 2023). For training DEAS and baselines, we use a batch size of 64 and train for 30K steps using Adam optimizer with a learning rate of 3×10^{-4} . For collecting rollouts, we use randomized environments using the object instance set A . For each task, we evaluate the model performance across 50 trials on five distinct evaluation scenes with 3 different evaluation seeds, totaling 150 rollouts. To test generalization capabilities, we evaluate the policy only on unseen object instances.

⁵<https://github.com/NVIDIA/Isaac-GR00T>

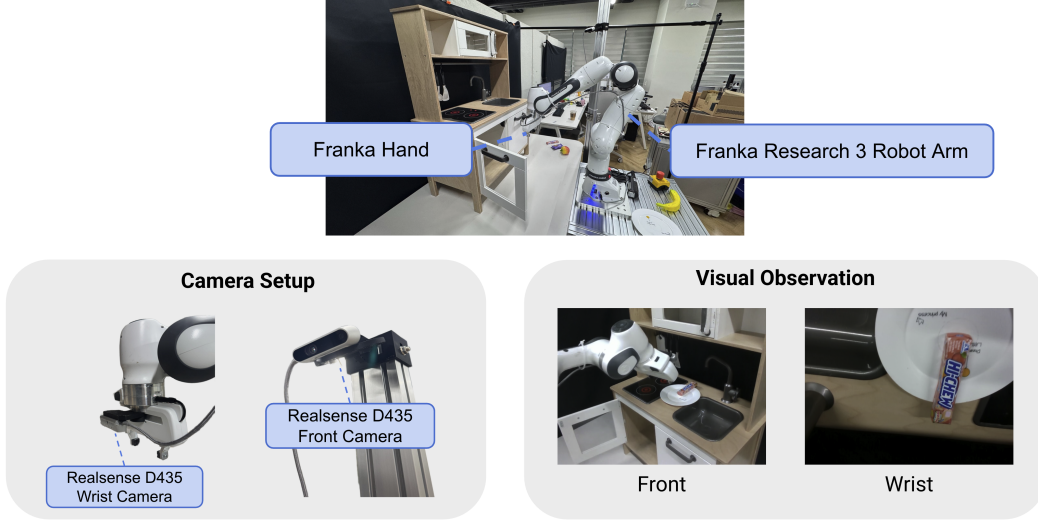


Figure 5: Real-robot platform.



Figure 6: Initialization points used for pick-and-place tasks.

A.2.2 REAL ROBOT EXPERIMENTS

Hardware platform We use Franka Research 3, a 7-DoF robotic arm, for our experiments. For visual perception, we utilize the dual camera with Intel RealSense D435i: a camera attached to the column next to the robot base to provide a global view, and a wrist-mounted camera for a close-range view. Teleoperated demonstrations are collected using an Oculus Quest 2, and we log time-synchronized RGB images, joint states, and gripper width for data collection. Demonstrations are recorded at 15 Hz. See Figure 5 for visual examples.

Task We evaluate the model performance on pick-and-place tasks from the countertop to the bottom cabinet, with three different objects: peach, milka, and hicew. Each object has different properties: peach is a rigid object with a relatively larger size that is easy to occlude, milka is a deformable object with a relatively smaller size that is easy to deform, and hicew is a hard object requiring precise grasping due to its small width. For collecting demonstrations, we use different initialization points (center, top, bottom, left, right) and collect one demonstration for each position (see Figure 6 for the initialization points used in our experiments). For accurate value function estimation, we manually label the reward function for each task. Specifically, we split the task into 4 stages: 1) moving to the countertop, 2) picking up the object, 3) moving to the target position, and 4) placing the object. For each stage, we label the reward function as 1 if the task is completed, and 0 otherwise, and we set the reward function as the negative number of uncompleted stages following Park et al. (2025a).

Implementation details Unless otherwise mentioned, we follow the same implementation details as in the RoboCasa Kitchen experiments. For selecting final actions, we use $N = 50$ candidates from the policy and use the same procedure for selecting the final action as in the RoboCasa Kitchen experiments.

A.3 HYPERPARAMETERS

We list the hyperparameters used in our OGBench experiments in Tables 5 and 6. For the BC coefficient α used for policy extraction, please refer to Table 7.

Table 5: DEAS hyperparameters for OGBench experiments.

Hyperparameter	Value
Gradient steps	1M (1M dataset), 2.5M (10M/100M dataset)
Optimizer	Adam (Kingma, 2015)
Learning rate	0.0003
Batch size	256 (1M dataset), 1024 (10M/100M dataset)
Actor MLP size	[512, 512, 512, 512] (1M dataset) [1024, 1024, 1024, 1024] (10M/100M dataset)
Critic MLP size	[256, 256, 256, 256] (1M dataset) [512, 512, 512, 512] (10M/100M dataset)
Value MLP size	[128, 128, 128, 128] (1M dataset) [256, 256, 256, 256] (10M/100M dataset)
Nonlinearity	GELU (Hendrycks & Gimpel, 2016)
Layer normalization	True
Target network update rate	0.005
Discount factor γ_1	0.9
Discount factor γ_2	0.995 (cube), 0.999 (scene, puzzle)
HL-Gaussian - Atoms	101
HL-Gaussian - σ	0.75
HL-Gaussian - Support range type	<i>data-centric</i> (cube), <i>universal</i> (scene, puzzle)
Flow steps	10
Critic ensemble size	2
Action sequence length H	4 (cube), 8 (scene, puzzle)
Expectile κ (DEAS)	0.9 (1M dataset), 0.95 (10M/100M dataset)
Double Q aggregation	$\min(Q_1, Q_2)$
Policy extraction hyperparameters	Table 7

Table 6: Baseline hyperparameters for OGBench experiments.

Hyperparameter	Value
Critic MLP size	[512, 512, 512, 512] (1M dataset) [1024, 1024, 1024, 1024] (10M/100M dataset)
Discount factor γ (FQL, n -step FQL)	0.99
Discount factor γ (QC-FQL)	0.995 (cube), 0.999 (puzzle)
Horizon reduction factor n	4 (cube), 8 (puzzle)
Policy extraction hyperparameters	Table 7
Levels (CQN-AS)	5
Bins (CQN-AS)	9
C51 - $\mathbf{v}_{\min}, \mathbf{v}_{\max}$ (CQN-AS)	-200, 0

Table 7: Policy extraction hyperparameters for OGBench experiments. Note that we apply Q-Normalization (Fujimoto & Gu, 2021) for actor loss, except cube-double tasks.

Task	FQL α	n -step FQL α	QC-FQL α	DEAS α
scene	3	1	3	3
cube-double	300	100	300	300.0
puzzle-3x3	3	1	1	3
cube-triple	3	1	1	1
puzzle-4x4	3	1	1	3
cube-quadruple	3	1	1	1

B EXTENDED RELATED WORK

Hierarchical RL and Options Framework Some Hierarchical RL works seek to address the challenges of long-horizon and sparse-reward tasks by reducing the effective horizon through learning value functions that consume multi-step actions (Kulkarni et al., 2016; Vezhnevets et al., 2017; Nachum et al., 2018; Ajay et al., 2021), usually combined with bi-level architectures. Among them, Options framework (Sutton et al., 1999; Stolle & Precup, 2002; Bacon et al., 2017) introduces formalization of higher-level actions that persist for multiple time steps with variable initiation/termination conditions, effectively reducing the planning horizon and facilitating more efficient learning. Our approach leverages the options perspective by treating action sequences as primitive options, enabling horizon reduction and improved value propagation without task-specific knowledge, explicit goal conditioning, or manual sub-task specification.

Reinforcement learning with VLAs Recent efforts have applied RL to VLA training (Zhang et al., 2024; Chen et al., 2025a; Zhang et al., 2025; Guo et al., 2025; Tan et al., 2025; Chen et al., 2025b; Li et al., 2025a), but most focus on on-policy online RL, which requires expensive interactions and cannot reuse transitions. A key limitation is that existing methods use single-step value functions $Q(s, a)$ for value learning, despite modern VLAs being designed to predict action sequences (Black et al., 2025; Bjorck et al., 2025; Intelligence et al., 2025). This mismatch between single-step value learning and multi-step action prediction limits the effectiveness of RL with VLAs. The most related work is CO-RFT (Huang et al., 2025), which applies chunked offline RL to VLA training, but differs from our approach in three key aspects: (1) CO-RFT uses actor-critic methods (Nakamoto et al., 2023) with single-step value functions while DEAS uses detached value learning with action sequences, (2) CO-RFT relies on human teleoperated expert demonstrations while we use small expert sets with large suboptimal rollouts, and (3) CO-RFT requires sophisticated transformer architectures while DEAS achieves improvements with simple MLP networks.

C LIMITATIONS AND FUTURE WORK

While DEAS demonstrates significant improvements over existing offline RL methods, several limitations and opportunities for future research remain. First, our current approach uses fixed action sequence lengths across different tasks, but the optimal sequence length varies significantly depending on task complexity. Future work should investigate adaptive mechanisms that can dynamically adjust action sequence lengths based on task requirements, potentially through adopting hierarchical policies (Kulkarni et al., 2016; Vezhnevets et al., 2017; Nachum et al., 2018). Second, while DEAS shows promising results on individual tasks, scaling to large-scale unified value functions remains a critical challenge for real-world deployment. DEAS currently trains reward models on 3-4 tasks simultaneously, but practical applications require learning from hundreds or thousands of diverse tasks. Future research should focus on developing scalable architectures and training procedures that can handle massive multi-task datasets while maintaining sample efficiency and avoiding catastrophic forgetting. Third, our method relies on distributional RL with fixed support ranges ($\mathbf{v}_{\min}$, $\mathbf{v}_{\max}$) and discretization parameters, which can significantly impact performance. The sensitivity to these hyperparameters limits the method’s robustness across different domains and reward scales. Future work should develop more robust frameworks that can automatically adapt to different reward distributions or provide principled ways to set these parameters.

D USE OF LARGE LANGUAGE MODELS

We acknowledge the use of large language models (LLMs) in preparing this manuscript. LLMs were employed solely to refine writing quality, including grammar correction, vocabulary suggestions, and typographical checks. All substantive ideas, analyses, and conclusions in this paper are entirely the work of the authors

E FULL EXPERIMENTAL RESULTS

We include the full experimental results in OGBench experiments in Table 8.

Table 8: Full offline RL Results in **30** OGBench tasks. * indicates the default task in each environment. We report the success rate (%) and 95% stratified bootstrap confidence interval over 4 runs.

Task	#Data	FQL	N-step FQL	QC-FQL	CQN-AS	DEAS
scene-play-singletask-task1-v0	1M	100 ± 0	100 ± 0	99 ± 0	2 ± 1	99 ± 1
scene-play-singletask-task2-v0		50 ± 7	4 ± 3	99 ± 1	1 ± 1	97 ± 1
scene-play-singletask-task3-v0		95 ± 2	78 ± 5	64 ± 8	0 ± 0	75 ± 6
scene-play-singletask-task4-v0*		3 ± 2	0 ± 0	68 ± 1	0 ± 0	65 ± 5
scene-play-singletask-task5-v0		0 ± 0	0 ± 0	35 ± 7	0 ± 0	45 ± 6
cube-double-play-singletask-task1-v0	1M	46 ± 4	17 ± 3	68 ± 4	7 ± 1	76 ± 3
cube-double-play-singletask-task2-v0*		10 ± 2	1 ± 0	47 ± 8	1 ± 1	51 ± 8
cube-double-play-singletask-task3-v0		9 ± 2	1 ± 1	40 ± 6	0 ± 1	47 ± 4
cube-double-play-singletask-task4-v0		1 ± 1	0 ± 0	8 ± 1	1 ± 1	8 ± 1
cube-double-play-singletask-task5-v0		2 ± 1	3 ± 1	44 ± 3	0 ± 0	57 ± 3
puzzle-3x3-play-singletask-task1-v0	1M	100 ± 0	89 ± 3	97 ± 1	1 ± 2	100 ± 0
puzzle-3x3-play-singletask-task2-v0		19 ± 4	40 ± 10	81 ± 12	0 ± 0	94 ± 5
puzzle-3x3-play-singletask-task3-v0		15 ± 2	14 ± 3	50 ± 11	0 ± 0	91 ± 3
puzzle-3x3-play-singletask-task4-v0*		35 ± 4	23 ± 3	31 ± 4	0 ± 0	91 ± 3
puzzle-3x3-play-singletask-task5-v0		47 ± 4	13 ± 3	50 ± 11	0 ± 0	96 ± 2
cube-triple-play-singletask-task1-v0	10M	31 ± 14	17 ± 5	100 ± 0	0 ± 0	98 ± 1
cube-triple-play-singletask-task2-v0*		9 ± 3	91 ± 4	92 ± 2	0 ± 0	95 ± 2
cube-triple-play-singletask-task3-v0		12 ± 5	0 ± 0	92 ± 2	0 ± 0	88 ± 3
cube-triple-play-singletask-task4-v0		0 ± 1	0 ± 0	59 ± 7	0 ± 0	45 ± 7
cube-triple-play-singletask-task5-v0		2 ± 1	0 ± 0	74 ± 4	0 ± 0	87 ± 5
puzzle-4x4-play-singletask-task1-v0	10M	54 ± 4	28 ± 5	66 ± 17	0 ± 0	92 ± 8
puzzle-4x4-play-singletask-task2-v0		24 ± 3	2 ± 1	80 ± 16	0 ± 0	42 ± 7
puzzle-4x4-play-singletask-task3-v0		36 ± 4	42 ± 7	69 ± 22	0 ± 0	99 ± 1
puzzle-4x4-play-singletask-task4-v0*		22 ± 2	28 ± 3	70 ± 17	0 ± 0	88 ± 4
puzzle-4x4-play-singletask-task5-v0		22 ± 4	3 ± 2	61 ± 19	0 ± 0	89 ± 6
cube-quadruple-play-singletask-task1-v0	100M	79 ± 6	70 ± 9	79 ± 7	0 ± 0	92 ± 5
cube-quadruple-play-singletask-task2-v0*		0 ± 0	97 ± 2	63 ± 7	0 ± 0	100 ± 0
cube-quadruple-play-singletask-task3-v0		6 ± 3	1 ± 1	33 ± 7	0 ± 0	62 ± 9
cube-quadruple-play-singletask-task4-v0		0 ± 0	13 ± 5	38 ± 7	0 ± 0	31 ± 7
cube-quadruple-play-singletask-task5-v0		0 ± 0	0 ± 0	12 ± 6	0 ± 0	35 ± 10